LAB5 예비 보고서

STM32_1 인터럽트와 PWM — FPGA 피아노·신호등 제어·ADC

과목 전자전기컴퓨터설계실험 2

제출일 2026년 10월 19일

성명 김태이

학번 2023440158

1 보고서 범위와 결론

10월 19일 실험(STM32_1)은 두 부분이다. NUCLEO-G474RE에서는 버튼 인터럽트, TIM6 주기 인터럽트, TIM3 PWM의 제공 예제를 실행한 뒤 과제3(LAB4의 과제1을 인터럽트로 다시 구현)과 과제4(버튼으로 PWM 비율 변경, C 디버거 사용)를 직접 작성한다. Combo II-DLD에서는 교재 실습 피아노, 신호등 제어, ADC를 시뮬레이션하고 보드에서 시연한다.

과제3·4의 코드는 한 파일에 작성하고 매크로 STM32_1_TASK로 골라 각각 Arm GCC로 빌드했다. 같은 소스를 PC에서 HAL 대역과 함께 실행하며 TIM6와 버튼 인터럽트를 흉내 내어 간격 순환, 길게 누름, 200 ms 잠금, 접점 떨림, PWM 비율 순환을 확인했다. FPGA 세 설계는 매뉴얼의 테스트벤치와 직접 작성한 파형 확인용 테스트벤치가 Icarus Verilog와 Vivado 시뮬레이터에서 모두 통과했다. 피아노는 bitstream까지 만들었고, 신호등과 ADC는 교재의 핀 표가 없어 합성까지만 했다.

LAB4에서 찾은 떨림 위험은 인터럽트 방식에도 그대로 남는다. 버튼 인터럽트는 상승 에지에서만 오므로, 200 ms보다 길게 눌렀다 뗄 때의 떨림이 새 누름으로 인정된다. 보드에서 이 현상이 나오면 TIM6로 버튼이 떨어진 것을 확인한 뒤에만 다음 누름을 받는 대비안으로 바꾼다.

항목	사전 검증	결과
과제3	Arm GCC 빌드, 호스트 4개 시나리오	1→2→4→8→1초
과제4	Arm GCC 빌드, 호스트 2개 시나리오	25→50→75→25 %
피아노	Icarus·Vivado TB 2개, bitstream	PASS, 타이밍 충족
신호등	Icarus·Vivado TB 2개, 합성	PASS, LUT 31개
ADC	Icarus·Vivado TB 2개, 합성	PASS, LUT 60개

여기서 PASS와 호스트 시뮬레이션 결과는 논리가 맞다는 뜻이다. 실제 인터럽트 지연, 버튼의 떨림, 계측기에서 본 PWM 파형, FPGA 핀 결선과 외부 장치의 동작은 포함하지 않으므로 각 장 끝의 표대로 보드에서 따로 확인한다.

2 실험 환경과 검증 방법

구분	사용한 것
MCU 보드	NUCLEO-G474RE (STM32G474RE, Cortex-M4)
MCU 프로젝트	Glaysia/ece2_474 v1.1.0
MCU 빌드	Arm GNU Toolchain 13.3.Rel1, CMake 3.24.2, Ninja 1.11.1
호스트 검증	MinGW-W64 gcc 9.5.0, HAL 대역 헤더
FPGA 보드	Combo II-DLD, xc7s75fgga484-1
FPGA 도구	Vivado 2025.2, Icarus Verilog 12.0

MCU 프로젝트는 LAB4와 같은 태그 v1.1.0을 그대로 쓴다. 매뉴얼은 4주 동안 한 프로젝트를 이어 쓰도록 되어 있어, LAB4 폴더를 lab5/stm32_lab으로 복사한 뒤 ece2_474_routine.c만 원본으로 되돌리고 이번 과제를 새로 작성했다. 과제3은 기본 Debug 프리셋으로, 과제4는 같은 설정에 -DSTM32_1_TASK=4만 더해 별도 폴더에 빌드했다.

호스트 시뮬레이션은 LAB4의 방식을 인터럽트로 넓혔다. 보드용 ece2_474_routine.c를 고치지 않고 PC용 gcc로 컴파일하며, 대역 헤더가 GPIO, HAL_GetTick, TIM 시작 함수와 __HAL_TIM_SET_COMPARE를 흉내 낸다. 시뮬레이터는 50 µs 단위로 시간을 진행하면서 TIM6가 시작되어 있으면 100 ms마다 HAL_TIM_PeriodElapsedCallback을, 버튼 값이 LOW에서 HIGH로 바뀔 때마다 HAL_GPIO_EXTI_Callback을 호출한다. 실제 하드웨어의 콜백 연결과 같은 함수가 불리므로 실습 코드의 판단 결과를 시각별로 볼 수 있다.

FPGA 실습은 매뉴얼의 설계 모듈과 테스트벤치를 실습별 폴더에 넣어 실행했다. 매뉴얼의 테스트벤치는 시간 매개변수를 줄여 전체 동작을 검사하지만 파형이 길어 값이 잘 보이지 않으므로, 핵심 구간만 보여 주는 tb_view.sv를 따로 작성했다. Vivado 파형 캡처는 한 단 폭에서 읽히도록 시간축을 반으로 잘라 두 줄로 배치했다.

3 소스 출처와 실행 스냅샷

항목	식별 정보
MCU 템플릿	https://github.com/Glaysia/ece2_474.git
태그·커밋	v1.1.0 · dd94f01047aebdece09b9620c66e9b179403e5c9
FPGA 코드	ece2_474 docs/FPGA_additional.md 3·4·7절
제출 저장소	https://github.com/tae2yi/UOS_ece2
LAB5 코드	57136f0 (보고서의 GitHub 주소가 가리키는 위치)
폴더	lab5/stm32_lab, stm32_1_hostsim, lab5_01_piano, lab5_02_traffic, lab5_03_adc

GitHub: https://github.com/tae2yi/UOS_ece2/tree/57136f0/lab5

템플릿에서 바꾼 파일은 이번에도 Core/Src/ece2_474_routine.c 하나다. 원본 대비 바뀐 내용 전체는 아래 diff에 있다. 호스트 시뮬레이션의 CSV, FPGA 시뮬레이션 로그, Vivado 리포트도 같은 저장소에 있다.

diff:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/report/pre/logs/stm32_routine.diff

STM32_1 인터럽트 구조와 제공 예제

폴링에서 인터럽트로

LAB4의 폴링 방식은 메인 루프가 쉬지 않고 버튼과 시각을 물어보는 구조였다. 인터럽트 방식에서는 사건이 생겼을 때 하드웨어가 CPU를 멈추고 정해진 함수(ISR)를 실행한 뒤 원래 자리로 돌려보낸다. Cortex-M4의 NVIC가 이 일을 맡으며, 여러 인터럽트가 겹치면 우선순위 숫자가 작은 쪽이 먼저 실행되고 실행 중인 낮은 우선순위의 ISR을 중단시킬 수도 있다[1].

템플릿은 CubeMX가 만든 IRQ 처리 함수에서 HAL 콜백을 거쳐 실습 함수로 연결해 둔다. 학생은 아래 표의 실습 함수만 채운다.

사건	연결 경로	우선순위
PC13 상승 에지	EXTI15_10 → HAL_GPIO_EXTI_Callback → button_isr_p7	7
TIM6 갱신(100 ms)	TIM6_DAC → HAL_TIM_PeriodElapsedCallback → tim6_isr_p4	4
SysTick(1 ms)	HAL_IncTick: HAL_GetTick 값 증가	15

TIM6의 우선순위(4)가 버튼(7)보다 높으므로, 버튼 ISR 실행 중에 TIM6 주기가 오면 TIM6 ISR이 끼어든다. 반대로 SysTick은 가장 낮은 15라서 ISR 안에서는 HAL_GetTick 값이 늘지 않는다. 버튼 ISR은 들어온 순간의 시각만 읽고 바로 끝나므로 문제가 없지만, ISR 안에서 HAL_Delay를 쓰면 영원히 기다리게 된다. 매뉴얼이 ISR 안의 HAL_Delay와 printf를 금지하는 이유 중 하나다.

타이머 설정 값

시스템 클록은 HSI 16 MHz를 PLL로 100 MHz로 만든 것이다(÷2 × 25 ÷ 2). APB1 분주가 1이므로 TIM3·TIM6에도 100 MHz가 들어간다. 타이머는 클록을 (Prescaler+1)로 나눠 카운터를 올리고, 카운터가 Period(ARR)에 이르면 0으로 돌아가며 갱신 사건을 낸다.

타이머	PSC	ARR	카운트 클록	주기
TIM6	9999	999	10 kHz	100 ms
TIM3	99	999	1 MHz	1 ms (1 kHz)

100 MHz ÷ 10000 ÷ 1000 = 10 Hz이므로 TIM6 ISR은 100 ms마다 한 번 불린다. TIM3은 1 µs마다 세어 1000칸을 한 주기로 쓴다.

제공 예제 세 가지

예제	init	동작
버튼	빈 상태	누를 때마다 LED 반전. 200 ms 안의 재호출 무시
TIM6	TIM6 시작	10번째 호출마다 LED 반전 → 1초 간격
PWM	PWM 시작, 비교값 500	PB4에 1 kHz, 50 % 파형

버튼 예제의 200 ms 비교는 접점 떨림 때문이다. 기계식 버튼은 누르는 순간 접점이 몇 번 붙었다 떨어지므로 상승 에지 인터럽트가 여러 번 올 수 있다. 첫 호출 뒤 200 ms 동안의 호출을 버리면 한 번 누름이 한 번으로 처리된다. 이 예제와 과제3·4의 잠금 코드는 같다.

STM32_1 과제3 ISR로 LED 간격 변경

요구 조건과 설계

· 버튼을 누를 때마다 LED 상태 변경 간격이 1 → 2 → 4 → 8 → 1초로 바뀐다. 한 번 누르면 한 번만 바뀐다.

· tim6_isr_p4()가 100 ms마다의 호출 횟수를 세어 선택한 간격마다 내장 LED(PA5)를 바꾼다.

· polling_routine()은 비우고 버튼과 LED 동작은 ISR에서만 처리한다.

두 ISR이 함께 쓰는 값은 간격 하나다. 간격을 ms 대신 TIM6 호출 횟수로 저장해 blink_ticks라 하고, 값은 10·20·40·80이다. 버튼 ISR은 이 값을 2배로 만들거나 80 다음에 10으로 돌리고, TIM6 ISR은 자기 카운터 count가 blink_ticks에 이르면 LED를 바꾼다.

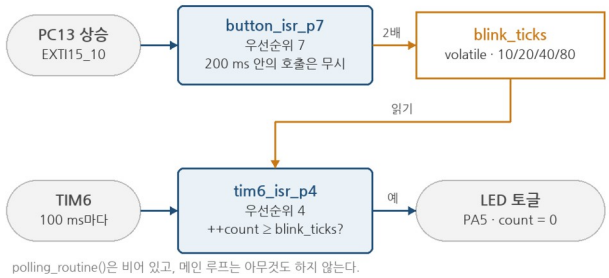


그림 1. 과제3의 처리 흐름. 버튼 ISR은 간격만 바꾸고, TIM6 ISR이 그 간격으로 LED를 토글한다.

변수	위치	쓰는 쪽 → 읽는 쪽
blink_ticks	파일 범위, volatile	버튼 ISR → TIM6 ISR
count	TIM6 ISR 안 static	TIM6 ISR만
last_ms	버튼 ISR 안 static	버튼 ISR만

blink_ticks는 두 함수가 함께 보므로 함수 밖에 두고 volatile을 붙였다. volatile은 컴파일러가 이 값을 레지스터에 담아 두고 재사용하지 않고 매번 메모리에서 읽고 쓰게 한다. 다만 여러 연산을 한 번에 끝내 주지는 않는다. 여기서는 버튼 ISR이 새 값을 계산한 뒤 32비트 저장 한 번으로 쓰고 TIM6 ISR은 읽기만 하므로, TIM6가 중간에 끼어들어도 이전 값 또는 새 값 중 하나를 읽을 뿐 깨진 값을 읽지 않는다. Cortex-M4에서 정렬된 32비트 읽기·쓰기는 한 번에 끝난다[1].

핵심 코드

```
/* 과제3: LED 상태 변경 간격을 TIM6 호출 횟수로 저장(10 = 1초).
* button_isr_p7()이 바꾸고 tim6_isr_p4()가 읽으므로 함수 밖에 둡니다.
*/
static volatile uint32_t blink_ticks = 10U;
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/stm32_lab/Core/Src/ece2_474_routine.c#L29-L32

```
static void button_isr_p7(void)
{
  static uint32_t last_ms = 0;
  uint32_t now_ms = HAL_GetTick();
  if (now_ms - last_ms < BUTTON_LOCK_MS)
  {
    return; /* 직전 누름 뒤 200 ms 안의 호출은 접점 떨림 */
  }
  last_ms = now_ms;
#if STM32_1_TASK == 3
  blink_ticks = (blink_ticks >= 80U) ? 10U : blink_ticks * 2U;
#else
  pwm_compare = (pwm_compare >= 750U)
        ? 250U : pwm_compare + 250U;
  __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_compare);
#endif
}
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/stm32_lab/Core/Src/ece2_474_routine.c#L87-L103

```
static void tim6_isr_p4(void)
{
#if STM32_1_TASK == 3
  static uint32_t count = 0;
  if (++count >= blink_ticks)
  {
    count = 0;
    HAL_GPIO_TogglePin(LED_ece2_GPIO_Port, LED_ece2_Pin);
  }
#endif
}
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/stm32_lab/Core/Src/ece2_474_routine.c#L109-L119

두 ISR 모두 판단과 대입 몇 개로 끝나 짧게 돌아온다. 과제4와 한 파일을 쓰므로 #if로 나눴고, ece2_474_init()에서는 과제3일 때 HAL_TIM_Base_Start_IT(&htim6)만 호출한다. 간격을 바꿀 때 count는 건드리지 않는다. 따라서 LAB4의 폴링 코드와 같이 새 간격은 직전 LED 변화 시점부터 센다. 8초에서 1초로 줄 때 count가 이미 10 이상이면 다음 TIM6 호출(100 ms 안)에서 바로 바뀐다.

예상 결과: 간격 순환

버튼을 4.55, 10.55, 22.55, 46.55초에 150 ms씩 누르는 시나리오를 실행했다. 누름 시각을 TIM6 주기(100 ms)의 중간에 두어 두 인터럽트가 같은 순간에 겹치지 않게 했다. LED가 바뀐 시각은 다음과 같다.

구간	LED가 바뀐 시각 (s)	간격
시작~4.55 s	1, 2, 3, 4	1 s
4.55~10.55 s	6, 8, 10	2 s
10.55~22.55 s	14, 18, 22	4 s
22.55~46.55 s	30, 38, 46	8 s
46.55 s 이후	47, 48, 49, 50, 51, 52	1 s

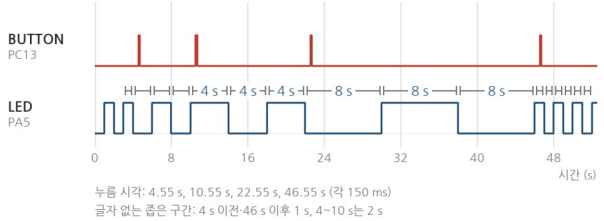


그림 2. 과제3 호스트 시뮬레이션. 누를 때마다 LED 변화 간격이 2, 4, 8초로 늘고 네 번째 누름에서 1초로 돌아간다.

원자료:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/stm32_1_hostsim/results/task3_cycle.csv

LAB4 과제1의 결과와 LED 변화 시각이 같다. 46.55초에 1초로 돌아갈 때는 46초의 변화 뒤 count가 5였으므로 47초에 바뀐다. 폴링 방식과 달리 메인 루프가 아무 일도 하지 않아도 같은 동작이 나온다.

예상 결과: 길게 누름과 200 ms 잠금

버튼 인터럽트는 상승 에지에서만 오므로, 6초 동안 누르고 있어도 인터럽트는 한 번이고 간격은 2초로 한 번만 바뀐다. 150 ms 간격으로 두 번 누르면 두 번째는 잠금 안이라 무시되고, 300 ms 간격이면 둘 다 인정된다.

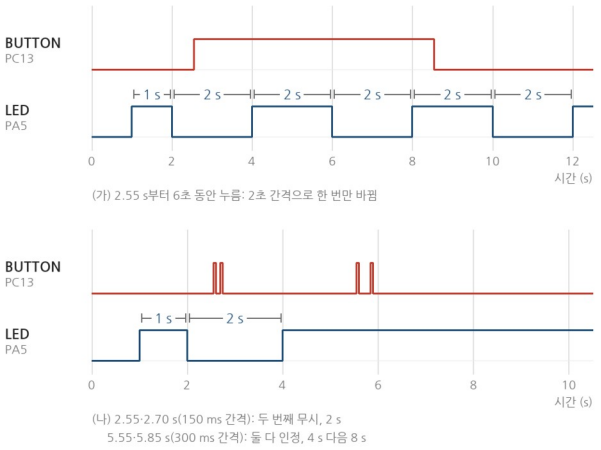


그림 3. (가) 6초 누름은 한 단계만 바뀐다. (나) 150 ms 간격의 두 번째 누름은 무시되고, 300 ms 간격은 두 번 모두 인정된다.

(나)에서 5.55초와 5.85초의 두 누름으로 간격이 4초를 거쳐 8초가 되었다. 다음 변화는 4초의 변화 뒤 8초인 12초라 10.5초까지의 그림에는 나오지 않는다. 사람이 빠르게 연타하면 200 ms 안의 누름이 버려질 수 있다는 점은 보드에서 같이 본다.

접점 떨림과 대비안

LAB4와 같은 떨림 모델(에지마다 0.4 ms 간격으로 5번)을 넣었다. 150 ms 누름은 누를 때와 뗄 때의 떨림이 모두 200 ms 잠금 안에 있어 한 번만 인정됐다. 1.5초 누름은 뗄 때 잠금이 이미 풀려 있어, 떨림 중 다시 HIGH가 되는 상승 에지가 새 누름으로 인정되고 간격이 4초에서 8초로 한 번 더 바뀌었다. EXTI가 하강 에지를 알려 주지 않으므로 LAB4의 대비안처럼 뗀 시각을 기록할 수 없다.

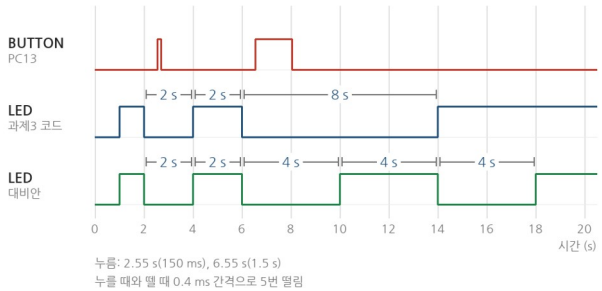


그림 4. 떨림이 있을 때. 과제3 코드는 두 번째 누름을 뗄 때 8초로 한 번 더 바꾸고, 대비안은 4초를 유지한다.

대비안은 armed 표시를 하나 둔다. 버튼 ISR은 armed일 때만 누름을 인정하고 armed를 내린다. TIM6 ISR은 100 ms마다 버튼 핀을 읽어 LOW이면 armed를 다시 올린다. 뗄 때의 떨림은 수 ms 안에 끝나 아직 TIM6가 LOW를 보기 전이므로 걸러진다.

```
static void tim6_isr_p4(void)
{
  if (HAL_GPIO_ReadPin(BUTTON_ece2_GPIO_Port, BUTTON_ece2_Pin)
    == GPIO_PIN_RESET)
  {
    armed = 1U;
  }
  ...
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/stm32_1_hostsim/variants/release_guard.c#L56-L62

버튼 ISR에는 if 조건에 !armed를 더하고 인정할 때 armed = 0U를 넣는다. 대가는 TIM6가 버튼이 떨어진 것을 볼 때까지(최대 100 ms) 다음 누름을 받지 못한다는 점이다. 이 떨림은 모델이며 NUCLEO 버튼 회로의 실제 떨림은 보드에서 확인할 항목이다. 그래서 매뉴얼의 구성과 같은 과제3 코드를 먼저 올리고, 길게 눌렀다 뗄 때 두 단계가 넘어가면 대비안으로 바꾼다. 대비안은 떨림이 없는 순환·길게 누름 시나리오에서 과제3 코드와 같은 결과를 냈다.

빌드 결과

```
$ cmake --preset Debug
$ cmake --build --preset Debug
RAM:    2656 B / 128 KB  (2.03 %)
FLASH: 26008 B / 512 KB  (4.96 %)
blink_ticks    .data  4 B
count, last_ms .bss   4 B each
```

로그:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/report/pre/logs/stm32_arm_build.txt

ece2_474_routine.c에서 경고는 없었다. 초기값 10이 있는 blink_ticks는 .data에, 0으로 시작하는 두 static 변수는 .bss에 놓였다.

보드에서 확인할 항목

확인 항목	예상 관찰
버튼 예제	누를 때마다 LED가 한 번 바뀜
TIM6 예제	버튼 없이 1초 간격으로 깜빡임
과제3 시작	리셋 후 1초 간격
누름 1~3회	2초, 4초, 8초 간격. 스톱워치로 확인

확인 항목	예상 관찰
누름 4회	다시 1초 간격
길게 누름	한 단계만 바뀜. 두 단계면 대비안 적용
빠른 연타	200 ms 안의 두 번째 누름은 무시

STM32_1 과제4 버튼으로 PWM 비율 변경

PWM의 원리

PWM은 주기를 고정하고 한 주기 안에서 HIGH인 시간의 비율(듀티비)을 바꾸는 출력이다. TIM3는 1 µs마다 0부터 999까지 세고 다시 0으로 돌아간다. PWM1 모드에서는 카운터가 비교값 CCR1보다 작은 동안 PB4가 HIGH, 그 뒤는 LOW이므로 듀티비는 CCR1 / 1000이다[3].

CCR1	HIGH 시간	듀티비	평균 전압
250	0.25 ms	25 %	약 0.83 V
500	0.50 ms	50 %	약 1.65 V
750	0.75 ms	75 %	약 2.48 V

평균 전압은 3.3 V에 듀티비를 곱한 값이다. LED를 PB4에 연결하면 1 kHz는 눈으로 깜빡임을 느끼기 어려울 만큼 빨라서 밝기가 듀티비에 따라 달라 보인다. 계측기에서는 주기 1 ms가 그대로이고 HIGH 폭만 바뀌어야 한다.

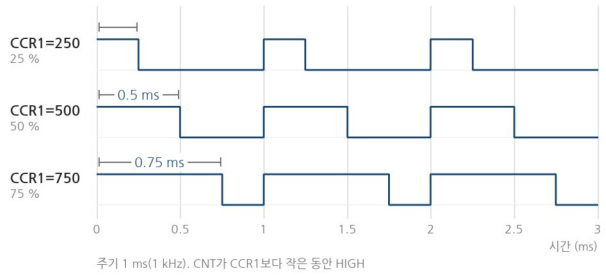


그림 5. 비교값에 따른 PB4의 예상 파형. 주기는 1 ms로 같고 HIGH 폭만 0.25, 0.5, 0.75 ms로 바뀐다.

설계와 핵심 코드

- 버튼을 누를 때마다 PWM 비율이 25 → 50 → 75 → 25 %로 바뀐다.
- htim3를 사용하고 PWM 시작은 ece2_474_init()에 작성한다.
- 과제3의 버튼 처리를 대신하므로 TIM6는 시작하지 않는다.

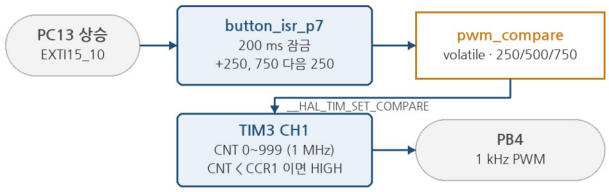


그림 6. 과제4의 처리 흐름. 버튼 ISR이 비교값을 바꾸면 TIM3 하드웨어가 PB4 파형을 계속 만든다.

```
static volatile uint32_t pwm_compare = 250U;


/* ece2_474_init() */
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_compare);
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/stm32_lab/Core/Src/ece2_474_routine.c#L33-L54

버튼 ISR은 과제3과 같은 함수의 #else 쪽이다. 잠금 검사 뒤 pwm_compare를 250 올리고 750 다음에는 250으로 돌린 다음 __HAL_TIM_SET_COMPARE로 CCR1에 쓴다. PWM 파형 자체는 TIM3 하드웨어가 만들므로 CPU는 버튼을 누를 때만 일한다.

CubeMX가 만든 TIM3 설정은 비교값 500에서 시작하므로, init에서 PWM을 시작한 직후 pwm_compare의 초기값 250을 써 25 %에서 시작한다. HAL_TIM_PWM_ConfigChannel이 CCR1 미리 읽기(preload)를 켜 두므로 새 비교값은 진행 중인 주기가 끝나는 갱신 시점(최대 1 ms 뒤)부터 적용된다[3]. 주기 중간에 바꿔도 파형이 잘리지 않는다.

예상 결과

1.05, 2.05, 3.05, 4.05초에 150 ms씩 누르는 시나리오에서 CCR1에 쓴 값은 250(0 s), 500(1.05 s), 750(2.05 s), 250(3.05 s), 500(4.05 s)이다. 0초의 500은 CubeMX 초기 설정이고 같은 순간 250으로 바뀐다.

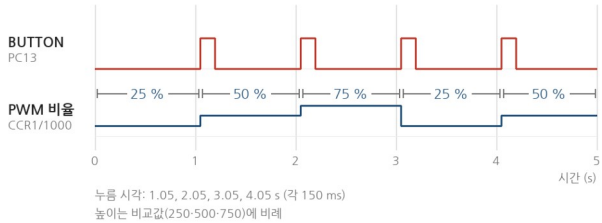


그림 7. 과제4 호스트 시뮬레이션. 누를 때마다 비율이 25→50→75→25→50 %로 바뀐다.

떨림 모델에서도 과제3과 같은 결과가 나왔다. 1.5초 누름을 뗄 때 75 %에서 25 %로 한 단계 더 넘어갔고, 대비안(TIM6를 함께 시작)에서는 75 %를 유지했다.

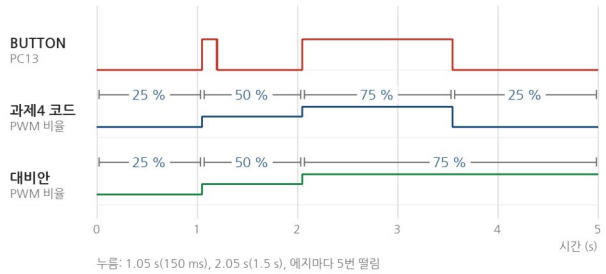


그림 8. 떨림이 있을 때 과제4 코드와 대비안. 3.55 s에 손을 뗄 때 과제4 코드만 한 단계 더 바뀐다.

C 디버거 사용 계획

순서	조작	확인할 것
1	pwm_compare 대입 줄에 F9	줄 왼쪽의 빨간 점
2	F5, main에서 멈추면 F5	버튼을 누르면 ISR에서 멈춤
3	Watch에 pwm_compare 추가	누르기 전 값 250
4	F10 한 줄 실행	500으로 바뀜, 다음 줄은 CCR1 쓰기
5	F10 한 번 더	htim3.Instance->CCR1 = 500
6	중단점 해제 후 F5	정상 실행에서 비율 순환

중단점에서 멈춰 있는 동안에도 TIM3은 마지막 비교값으로 PWM을 계속 낸다. 타이머는 CPU와 따로 동작하는 하드웨어이기 때문이다. 반대로 멈춘 시간 동안 HAL_GetTick이 늘지 않을 수 있으므로, 디버거로 멈춘 상태의 시간은 측정에 쓰지 않는다. Watch에서 CCR1이 바로 보이지 않으면 Variables의 htim3를 펼쳐 Instance 레지스터를 본다.

빌드 결과와 보드에서 확인할 항목

```
$ cmake ... -DCMAKE_C_FLAGS=-DSTM32_1_TASK=4
RAM:    2648 B / 128 KB  (2.02 %)
FLASH: 26324 B / 512 KB  (5.02 %)
pwm_compare    .data  4 B
```

로그:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/report/pre/logs/stm32_arm_build.txt

확인 항목	예상 관찰
PWM 예제	PB4 1 kHz, HIGH 폭 0.5 ms
과제4 시작	HIGH 폭 0.25 ms (25 %)
누름 1~3회	0.5 → 0.75 → 0.25 ms, 주기 1 ms 유지
길게 누름	한 단계만 바뀜. 두 단계면 대비안 적용
디버거	위 표의 값 변화와 일치
계측기	PB4와 GND 연결. 주파수·듀티비 표시 기록

FPGA-01 피아노

회로 목적과 구조

버튼 8개 중 하나를 누르면 그 음의 주파수로 피에조를 울린다. 피에조는 사각파의 주파수대로 진동하므로, 원하는 주파수 f의 반주기마다 출력을 반전하면 된다. 50 MHz 클록에서 반주기는 50,000,000 / (2f) 클록이다.

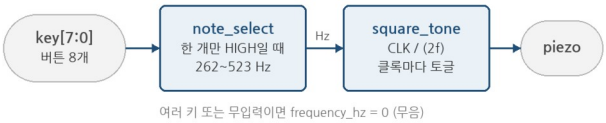


그림 9. FPGA-01 블록 흐름. 설계 top은 piano.

key	음	f (Hz)	반주기 클록	실제 (Hz)
8'h80	도	262	95,419	262.00
8'h40	레	294	85,034	294.00
8'h20	미	330	75,757	330.00
8'h10	파	349	71,633	349.00
8'h08	솔	392	63,775	392.00
8'h04	라	440	56,818	440.00
8'h02	시	494	50,607	494.00
8'h01	높은 도	523	47,801	523.00

나눗셈의 나머지를 버리므로 실제 주파수는 목표보다 아주 조금 높지만 차이는 0.01 Hz 이하다. 두 개 이상 누르거나 아무것도 누르지 않으면 note_select가 0을 내고 square_tone은 출력을 0으로 둔다. 다른 음으로 바뀌면 카운터와 출력을 0부터 다시 시작한다.

```
module square_tone #(parameter CLK_HZ=50_000_000)(
  input wire clk_50mhz, rst_p,
  input wire [31:0] frequency_hz,
  output reg tone
);
  reg [31:0] count, previous_hz;
  wire [31:0] half_cycles = frequency_hz == 0 ? 1 :
              CLK_HZ / (2 * frequency_hz);
  always @(posedge clk_50mhz) begin
    if(rst_p) begin count<=0; tone<=0; previous_hz<=0; end
    else if(frequency_hz==0 || frequency_hz!=previous_hz) begin
      count<=0; tone<=0; previous_hz<=frequency_hz;
    end
    else if(count >= half_cycles-1) begin
      count<=0; tone<=~tone;
    end
    else count<=count+1;
  end
endmodule
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/lab5_01_piano/src/square_tone.v#L2-L20

파일의 역할

파일	역할
src/note_select.v	버튼 → 주파수(Hz) 표
src/square_tone.v	반주기마다 토글하는 사각파
src/piano.v	설계 top. 두 모듈 연결
sim/tb.sv	매뉴얼 TB. 8음 타이밍, 무입력, 동시 입력, 리셋
sim/tb_view.sv	파형 확인용. 도·미·높은 도, 동시 입력

파일	역할
constraints/piano.xdc	버튼 8개·리셋·피에조·50 MHz 클록

시뮬레이션 결과

```
[Icarus] PASS piano: all 8 tones, silence, multiple keys, reset
[Vivado] PASS piano: ... (4796 ns)
[Vivado] PASS piano view: 262/330/523 Hz tones,
    6 rising edges (2270 ns)
```

로그:
https://github.com/tae2yi/UOS_ece2/tree/57136f0/lab5/report/pre/logs

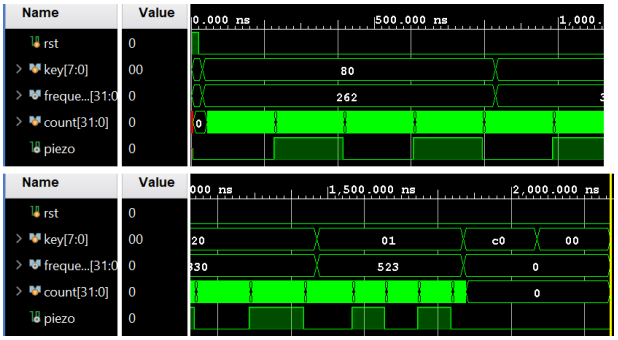


그림 10. FPGA-01 Vivado 파형(tb_view, CLK_HZ=10000). 두 줄 배치. key가 80·20·01일 때 반주기 19·15·9클록으로 토글하고, c0(두 키)와 00에서는 frequency_hz=0, piezo=0.

TB는 CLK_HZ를 10000으로 줄여 반주기를 수십 클록으로 만든다. 그림에서 음이 높을수록 piezo의 주기가 짧아지고, key가 바뀌는 순간 count가 0으로 돌아가는 것이 보인다. 매뉴얼 TB는 여덟 음 모두에서 4주기 동안 매 클록 출력을 검사한다.

구현 결과

항목	값
LUT	646개 (square_tone 631), 레지스터 43개
타이밍	50 MHz에서 WNS +14.043 ns, 위반 없음
DRC	오류 0, 경고 1 (CFGBVS 미설정)
bitstream	lab5_01_piano/bitstream/piano.bit

LUT 대부분은 square_tone의 CLK_HZ / (2 × frequency_hz)다. frequency_hz가 32비트 변수라 합성기가 나눗셈 회로를 만든다. 반주기 값이 여덟 개뿐이므로 note_select에서 반주기를 바로 내면 나눗셈이 없어진다. 동작에는 문제가 없고 타이밍도 충족해 매뉴얼 코드를 그대로 쓴다.

핀 배치와 보드에서 확인할 항목

신호	핀	근거
clk_50mhz	B6	LAB3과 같음
rst_p	K4	LAB3 리셋 버튼
key[7:0]	N8 N4 N1 P6 N6 L5 J2 K2	조교 키패드 예제 keys[1..8]
piezo	Y21	LAB3-03 피에조

버튼 핀은 조교 저장소의 키패드 시험 예제(Combo II-DLD 설명서의 버튼 표)를 따랐다. 교재의 piano.zip 핀 표와 다르면 교재를 따른다.

확인 항목	예상 관찰
버튼 1~8	도~높은 도. 오른쪽으로 갈수록 높은 음
두 버튼 동시	무음
버튼 뗌	무음
리셋	무음, 다시 누르면 정상

FPGA-02 신호등 제어

회로 목적과 구조

네 방향 교차로의 차량 신호와 보행자 신호를 1초 단위로 바꾼다. 한 방향이 13초(초록 11초, 노랑 2초)를 쓰고 다음 방향으로 넘어간다. flicker 입력이 HIGH이면 모든 차량 신호가 노랑, 보행자 신호가 빨강으로 함께 점멸한다.

1초를 세는 회로는 새 클록을 만들지 않고, 1초에 한 클록만 HIGH인 tick을 만들어 다른 레지스터의 갱신 조건으로 쓴다. 모든 레지스터가 같은 50 MHz 클록에서 움직이므로 LAB2에서 배운 분주 클록보다 타이밍 분석이 단순하다[2]. blink는 매초 앞의 0.5초 동안 HIGH다.

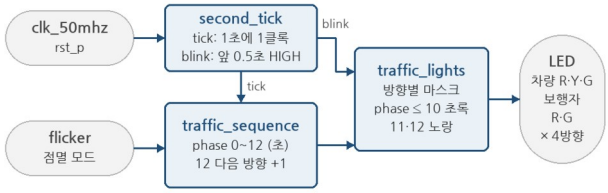


그림 11. FPGA-02 블록 흐름. 설계 top은 traffic.

phase (초)	차량	보행자(대응 방향)
0~3	초록	초록
4~10	초록	초록 점멸(blink)
11~12	노랑	빨강
flicker=1	모두 노랑 점멸	모두 빨강 점멸

선택된 방향이 아닌 차량 신호는 빨강이다. 방향 0의 차량 마스크는 4'b1000, 그 보행자 마스크는 4'b0100이며 방향이 바뀔 때마다 한 비트씩 오른쪽으로 이동한다(방향 3의 보행자는 4'b1000). 좌회전 신호 led_left는 초록과 같은 값을 낸다.

```
module traffic_sequence(
  input wire clk_50mhz,rst_p,tick,flicker,
  output reg [1:0] direction,
  output reg [3:0] phase
);
  always @(posedge clk_50mhz) begin
    if(rst_p || flicker) begin direction<=0; phase<=0; end
    else if(tick)
      if(phase==12) begin
        phase<=0; direction<=direction+1'b1;
      end
      else phase<=phase+1'b1;
  end
endmodule
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/lab5_02_traffic/src/traffic_sequence.v#L3-L16

flicker가 HIGH인 동안 방향과 단계는 0에 머문다. 점멸을 끄면 방향 0의 처음부터 다시 시작한다. second_tick은 flicker로 초기화되지 않으므로 끈 직후 첫 tick까지는 1초보다 짧을 수 있다.

시뮬레이션 결과

```
[Icarus] PASS traffic: full cycle, pedestrian blink,
    flicker, reset (2186 ns)
[Vivado] PASS traffic: ... (2186 ns)
[Vivado] PASS traffic view: direction 0 cycle,
    direction 1 start, flicker (820 ns)
```

로그:
https://github.com/tae2yi/UOS_ece2/tree/57136f0/lab5/report/pre/logs

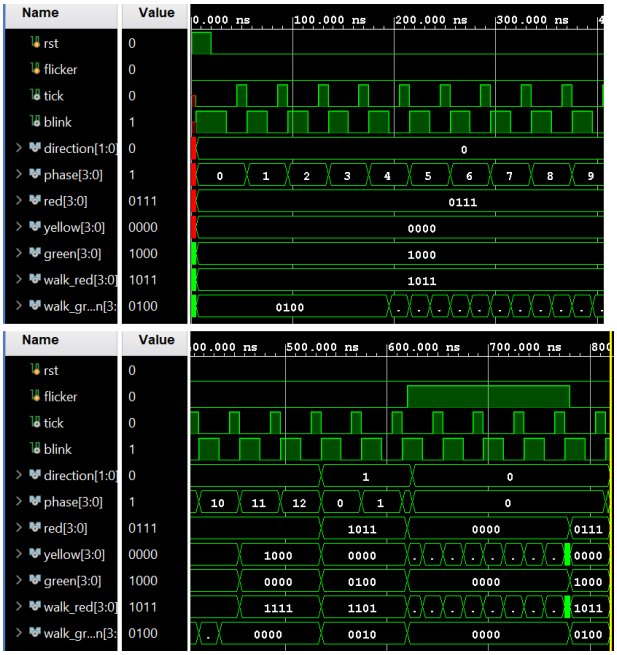


그림 12. FPGA-02 Vivado 파형(tb_view, CLK_HZ=4라 40 ns가 1초). 방향 0의 0~12단계, 방향 1의 시작, 600 ns 부근부터의 점멸 모드.

파형에서 phase 0~3은 walk_green=0100이 계속 켜지고, 4~10은 blink에 따라 0100과 0000을 오가며, 11~12에서는 yellow=1000, walk_red=1111이 된다. 방향 1로 넘어가면 green=0100, walk_green=0010이다. 점멸 모드에서는 red·green이 0이고 yellow와 walk_red가 함께 1111과 0000을 오간다.

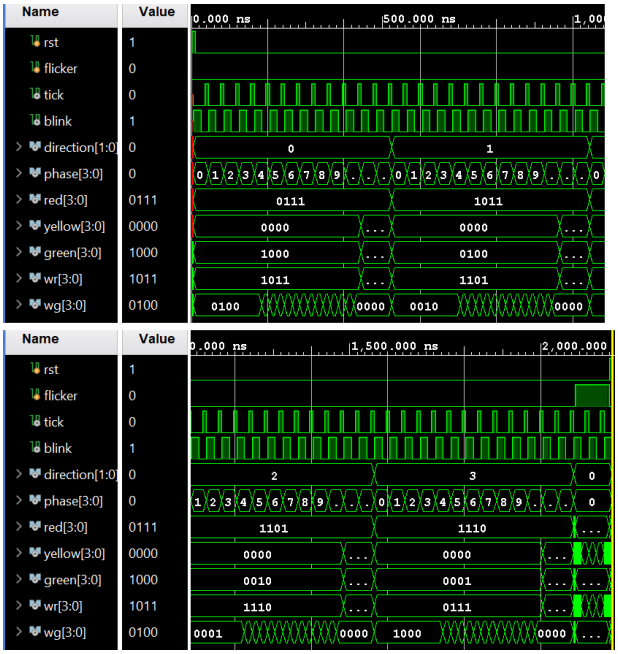


그림 13. FPGA-02 Vivado 전체 파형(tb, 2186 ns, 두 줄 배치). 네 방향의 순환, 점멸 모드, 리셋.

합성 결과와 보드에서 확인할 항목

XDC 없이 합성만 하면 LUT 31개, 레지스터 32개, 입출력 27개다. 출력이 차량 16개(좌회전 포함)와 보행자 8개로 24개라 보드의 LED 8개로는 모자란다. 교재의 신호등 실습이 어떤 표시 장치에 연결되는지와 그 핀 표를 실험 당일 확인한 뒤 XDC를 작성한다.

확인 항목	예상 관찰
리셋 뒤	방향 0 초록, 대응 보행자 초록
4초부터	보행자 초록이 0.5초씩 점멸
11초부터	방향 0 노랑, 보행자 빨강
13초	방향 1로 이동
flicker	노랑·보행자 빨강 동시 점멸
LED 극성	반대로 켜지면 교재의 극성 확인

FPGA-03 ADC

회로 목적과 구조

보드의 직렬 ADC 칩에 변환 명령을 보내고 변환 결과 12비트를 받아 LED에 표시한다. FPGA는 SCLK, CS, DIN을 만들고 DOUT을 받는 주인 역할이다. CS를 LOW로 둔 16클록 동안 명령 16비트를 내보내면서 동시에 결과 16비트를 받는다. 다음 주차의 SPI와 같은 구조다.

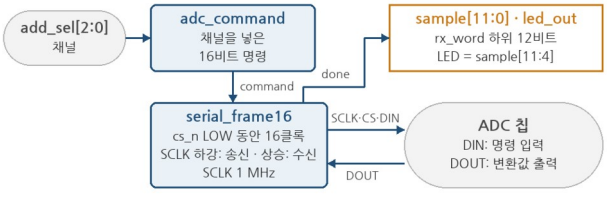


그림 14. FPGA-03 블록 흐름. 설계 top은 adc.

비트	값	뜻 (원본 예제의 구성)
15~13	101	WR, SEQ, 고정 비트
12~10	add_sel	채널 주소
9~8	11	전원 모드
7	0	SHADOW
6~4	111	고정·범위·코딩
3~0	0000	사용 안 함

매뉴얼은 원본 예제의 명령 비트 구성을 유지했다고 밝히며, 채널 2의 명령은 16'hAB70이다. 각 비트의 정확한 의미와 채널을 바꾼 뒤 결과가 몇 프레임 늦게 나오는지는 보드의 ADC 칩 설명서로 확인할 항목이다.

serial_frame16은 HALF_CYCLES 클록마다 SCLK를 뒤집는다. 기본값 25에서 SCLK는 50 MHz / 50 = 1 MHz다. SCLK 하강에서 다음 명령 비트를 내고 상승에서 DOUT을 받는다. 한 프레임이 끝나면 다음 클록에 바로 다음 프레임을 시작하며, 50 MHz 시뮬레이션에서 CS 하강 간격은 16.04 µs였다. 1초에 약 6만 2천 번 값을 새로 받는다.

```
  always @(posedge clk_50mhz) begin
    if(rst_p) begin start<=0; sample<=0; data_valid<=0; end
    else begin
      start<=!busy && !start;
      data_valid<=done;
      if(done) sample<=rx_word[11:0];
    end
  end
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/57136f0/lab5/lab5_03_adc/src/adc.v#L18-L25

start는 직렬 회로가 쉬고 있을 때 한 클록만 1이 되어 다음 프레임을 시작한다. done이 오면 받은 워드의 하위 12비트를 sample에 저장하고 data_valid를 한 클록 올린다. LED에는 상위 8비트 sample[11:4]를 낸다.

시뮬레이션 결과

```
[Icarus] PASS ADC: 3 serial frames, command bits,
    sample, reset (2016 ns)
[Vivado] PASS ADC: ... (2016 ns)
[Vivado] PASS ADC view: command ab70, sample 5c3,
    led 5c (800 ns)
```

로그:
https://github.com/tae2yi/UOS_ece2/tree/57136f0/lab5/report/pre/logs

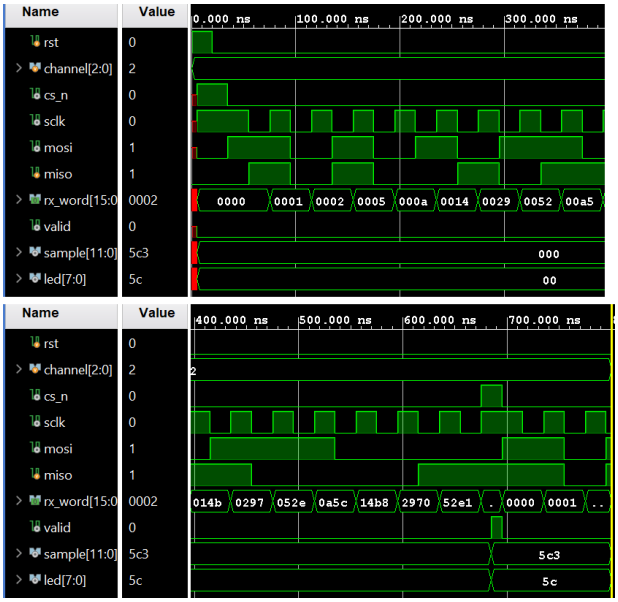


그림 15. FPGA-03 Vivado 파형(tb_view, HALF_CYCLES=2). 한 프레임 동안 mosi로 1010_1011_0111_0000을 내고 miso로 A5C3을 받아 sample=5c3, led=5c. 마지막 rx_word=a5c3 칸은 좁아 "."로 보인다.

파형 확인용 TB의 ADC 모델은 SCLK 상승에서 mosi를 받는다. 처음에는 CS가 HIGH가 되면 받지 않도록 했는데, 마지막 상승 에지와 CS 해제가 같은 클록에 일어나 15비트만 받았다. 매뉴얼 TB처럼 비트 수로 16에서 끊도록 고쳤다. 설계가 아닌 TB 모델의 문제였지만, 실제 ADC 칩도 마지막 비트를 CS 해제 직전에 읽을 수 있는지 설명서의 시간 조건으로 확인해야 한다는 뜻이다.

합성 결과와 보드에서 확인할 항목

XDC 없이 합성하면 LUT 60개, 레지스터 72개다. ADC 칩의 SCLK·CS·DIN·DOUT 핀과 채널 선택 입력은 교재 adc.zip의 핀 표로 XDC를 작성한다.

확인 항목	예상 관찰
입력 0 V	LED 모두 꺼짐에 가까움
입력 증가	LED 값이 이진수로 커짐
최대 입력	LED 모두 켜짐에 가까움
채널 변경	다른 입력의 값으로 바뀜
오실로스코프	CS LOW 16 µs, SCLK 1 MHz 16펄스

4 예상 결과 정리와 실험 계획

이번 주의 MCU와 FPGA 실습에는 공통점이 있다. 과제 3의 TIM6 ISR은 100 ms마다 오는 사건을 세어 1~8초를 만들고, 신호등의 second_tick도 클록을 세어 1초에 한 번 tick을 낸다. 둘 다 느린 동작을 위해 새 클록을 만들지 않고, 빠른 기준 신호의 횟수로 판단한다. PWM도 같은 원리다. TIM3 카운터와 비교값으로 HIGH 폭을 정하며, 피아노의 square_tone은 카운터로 반주기를 정한다.

순서	할 일	기록할 것
1	버튼·TIM6·PWM 예제	LED 영상, PB4 파형
2	과제3 다운로드	네 간격을 스톱워치로 측정
3	길게 누름 시험	간격이 몇 단계 바뀌는지
4	과제4 다운로드	세 비율의 PB4 파형 캡처
5	디버거 실습	Watch 화면 캡처
6	피아노 bitstream	버튼별 소리 영상
7	신호등·ADC	교재 핀 표로 XDC 작성 후 시연

실험 후 보고서에서는 위 측정값을 이 보고서의 예상 시각, 예상 HIGH 폭과 비교하고, 길게 누름에서 떨림 문제가 실제로 나왔는지와 그때의 대응을 적는다. 신호등과 ADC는 실제 핀 배치와 함께 결과를 정리한다.

참고 문헌

[1] Joseph Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed., Newnes, 2013. ISBN 978-0-12-408082-9.

[2] M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed., Pearson, 2018. ISBN 978-0-13-454989-7.

[3] STMicroelectronics, STM32CubeG4 MCU Firmware Package (stm32g4xx_hal_tim.c, stm32g4xx_hal_gpio.c). https://github.com/STMicroelectronics/STM32CubeG4